

SE446: Big Data Engineering

Week 10A: Apache Kafka — Distributed Messaging

Prof. Anis Koubaa

SE 446
Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Instructor Notes

Welcome everyone to Week 10A. Today we are entering a completely new territory. Until now, every single tool we have used — HDFS, MapReduce, Hive, Spark — has assumed that your data already exists somewhere on disk. You load a CSV, you read a Parquet file, you query a table. But what happens when data does not sit still? What happens when events arrive continuously, thousands per second, and you need to react immediately? That is what today is about. We are going to learn Apache Kafka, which is the most widely used distributed messaging platform in the industry. Think of Kafka as the central nervous system of a modern data architecture. It sits between the systems that produce data and the systems that consume data. And here is why this matters for you specifically: in Session 10B, we are going to connect Kafka directly to Spark Structured Streaming. So today you learn the highway; next time, you build the cars that drive on it. Let us get started.

Today's Agenda

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Instructor Notes

Here is our roadmap. We have seven sections today. First, we will talk about why we even need Kafka — what goes wrong when you only have batch processing. Second, we will dive deep into Kafka's architecture, and I am going to explain the why behind every design choice, not just definitions. Third, we will follow a single message end to end through the entire system, from the moment a crime report is filed to the moment Spark processes it. Fourth, we will look at fault tolerance — what happens when a server crashes. Fifth, a quick comparison with RabbitMQ so you understand when to use what. Sixth, hands-on practice with CLI commands and Python code. And seventh, design decisions you need to make when setting up Kafka topics. Notice how everything connects: the architecture explains the concepts, the message journey ties them together, and the practice section lets you use them. By the end, you will have a complete mental model of how Kafka works.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Learning Objectives

By the end of this session you will be able to:

- 1 Explain **why** batch processing alone is insufficient and what problem Kafka solves
- 2 Describe Kafka's architecture: brokers, topics, partitions, offsets, replication
- 3 Trace a single message's journey from producer to consumer
- 4 Understand consumer groups and **why** they exist
- 5 Publish and consume messages using CLI and Python
- 6 Connect Kafka concepts to the Spark Streaming pipeline we build next session

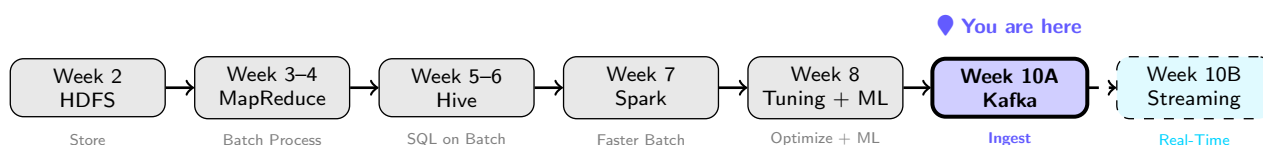
Where This Fits in Our Spark Journey

Until now, all our Spark jobs assumed **data already exists** on disk. Today we learn the **ingestion layer** that feeds real-time data into Spark. Next session (10B), we connect them: Kafka → Spark Structured Streaming.

Instructor Notes

Let me walk through these six objectives because each one maps to a section of today's lecture. Objective one is about motivation — why batch is not enough. This is not a theoretical question; I am going to show you three real scenarios where batch processing literally fails. Objective two is architecture, and I want to emphasize something: I am not going to throw a table of definitions at you. We are going to build the architecture from first principles, asking what problem does each piece solve. Objective three, we trace one message end to end — this is where everything clicks together. Objective four, consumer groups, which is how you scale reads. Objective five, you will actually produce and consume messages using both the command line and Python. And objective six is the bridge to next session: everything you learn today feeds directly into `spark.readStream.format("kafka")` which you will write in Session 10B. Notice the block at the bottom — this is the missing piece in our Spark journey. We have been assuming data exists. Today we learn how it gets there.

The Big Data Journey — Where Are We?



What We Built So Far

Store (HDFS) → Process (MR/Spark)
→ Query (Hive/SQL) → Optimize + ML

The Missing Piece

All of these assume data **already sits** on disk. What if it **arrives non-stop**?

Instructor Notes

Look at this timeline. You started the semester with HDFS — learning how to store data across a cluster. Then MapReduce — processing it the hard way, writing mappers and reducers. Then Hive gave you SQL on top of that. Spark made everything faster. Last session we added machine learning. Now look at where we are: Week 10A, Kafka. Notice the label underneath says “Ingest.” That is a very deliberate word. Every tool we have used so far starts with: “Okay, your data is in HDFS, now let us process it.” But nobody asked the question: how did the data get into HDFS in the first place? What if the data is not a static file but a continuous stream of events — crime reports being filed every second, sensor readings every millisecond, user clicks every moment? That is the missing piece. Kafka is the ingestion layer. It sits before everything else. And look at the dashed arrow to Week 10B — next session we connect Kafka to Spark Structured Streaming, completing the full pipeline from real-time ingestion to real-time processing.

The Problem: Batch Is Too Late

Consider these three scenarios. Could you solve them with a nightly Spark batch job?

Fraud Detection

Stolen card used
at 2:03 AM.
Batch job runs
at 6:00 AM.

4 hours of fraud.

Surge Pricing

Concert ends;
10K requests.
Hourly batch
sees it at 3:00.

Passengers left.

IoT Alert

Server room hits 90°C.
Alert must fire
at 2:03 PM.

Not tomorrow.

The Core Distinction

Batch processing answers: “*What happened?*”

Real-time processing answers: “*What is happening right now?*”

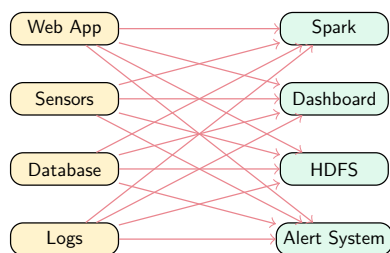
We need a system that can **receive events the instant they happen** and make them available to consumers **immediately**.

Instructor Notes

I want you to think about these three scenarios carefully. Left box: fraud detection. A stolen credit card is used at 2:03 AM. Your Spark batch job runs at 6:00 AM. That is four hours of fraudulent transactions that you could have stopped. The bank loses money, the customer is angry, and your system failed. Middle box: surge pricing for a ride-sharing company. A concert ends and suddenly 10,000 people need a ride. Your hourly batch job does not see this demand spike until the next hour. By then, those passengers have already left — they took a taxi, they walked, they are gone. Right box: an IoT sensor in a server room hits 90 degrees Celsius. If your monitoring system only checks once a day, your servers melt. The alert needs to fire at 2:03 PM, not the next morning. Now read the alert block at the bottom. This is the core distinction I want you to internalize. Batch processing tells you what happened. Real-time processing tells you what is happening right now. These are fundamentally different questions, and they require fundamentally different infrastructure. That infrastructure is Kafka.

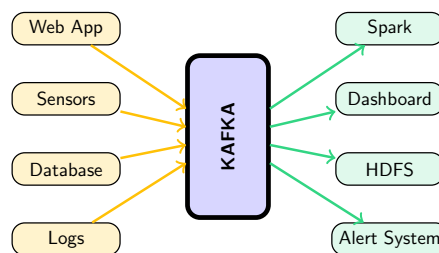
The Integration Nightmare (Before Kafka)

Imagine you have 4 source systems and 4 target systems. Without a central hub:



$4 \times 4 = 16$ connections

Each with its own protocol, format, error handling



$4 + 4 = 8$ connections

One protocol, one format, one system

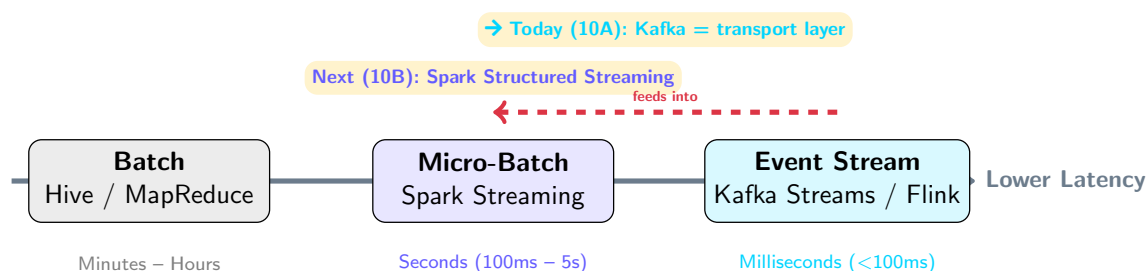
Why This Matters

LinkedIn had this exact problem in 2011: dozens of systems needed to exchange data. Point-to-point connections became unmanageable. They built Kafka as a **central nervous system** — every system publishes to Kafka, every system reads from Kafka. **Producers and consumers never need to know about each other.**

Instructor Notes

This slide explains the second reason Kafka exists, and it is just as important as real-time processing. Look at the left diagram. You have four source systems on the left — a web app, sensors, a database, and log files. And four target systems on the right — Spark, a dashboard, HDFS, and an alert system. Without a central hub, every source needs a direct connection to every target. That is 4 times 4 equals 16 connections. Each one has its own protocol, its own data format, its own error handling. Now imagine you add a fifth source. That is 4 more connections. A sixth target? Another 6. This grows quadratically and becomes a maintenance nightmare. Now look at the right diagram. Put Kafka in the middle. Every source sends data to Kafka. Every target reads from Kafka. That is 4 plus 4 equals 8 connections. Add a new source? One new connection. Add a new target? One new connection. This is what we call decoupling. LinkedIn had exactly this problem in 2011 — dozens of internal systems exchanging data through spaghetti connections. They built Kafka to solve it, and then open-sourced it. This is why Kafka was invented: not just for speed, but for architectural sanity.

The Paradigm Spectrum — Where Kafka Sits



How These Two Sessions Connect

Kafka = the highway that moves data between systems (today).

Spark Structured Streaming = the engine that processes data on that highway (next session).

In Session 10B, you will write: `spark.readStream.format("kafka")` — connecting directly to what you learn today.

Instructor Notes

Pay attention to this spectrum because it tells you where each tool sits. On the far left, batch processing — Hive and MapReduce. Latency is minutes to hours. You run a job, you wait, you get results. In the middle, micro-batch — this is Spark Structured Streaming, which processes data in small batches every few seconds. On the far right, true event streaming — Kafka Streams and Apache Flink, which process events one by one with sub-100-millisecond latency. Now here is the important part. Notice the two labels at the top. Today, we are learning Kafka, which is the transport layer. It moves data. It does not process it. Next session, we learn Spark Structured Streaming, which sits in the micro-batch zone and processes the data that Kafka delivers. See the dashed red arrow that says “feeds into”? That is the relationship. Kafka is the highway. Spark is the car. You need both. And at the bottom, I have written the exact line of code you will use next session: `spark.readStream.format("kafka")`. Everything you learn today about topics, partitions, and offsets will show up in that one line of code.

Kafka in One Sentence

Imagine you are the CTO of a ride-sharing company.

Every second, **100,000 events** happen: ride requests, GPS pings, payments, driver status changes.

You need a system that can:

- 🚀 **Receive** all of them without dropping any
- 💾 **Store** them reliably on disk (not just in memory)
- 👥 Let **10 different teams** read them independently, at their own pace
- ⚡ Do all of this at **millisecond latency**

That system is Apache Kafka.

Definition

Kafka is a **distributed, fault-tolerant, persistent event streaming platform**.

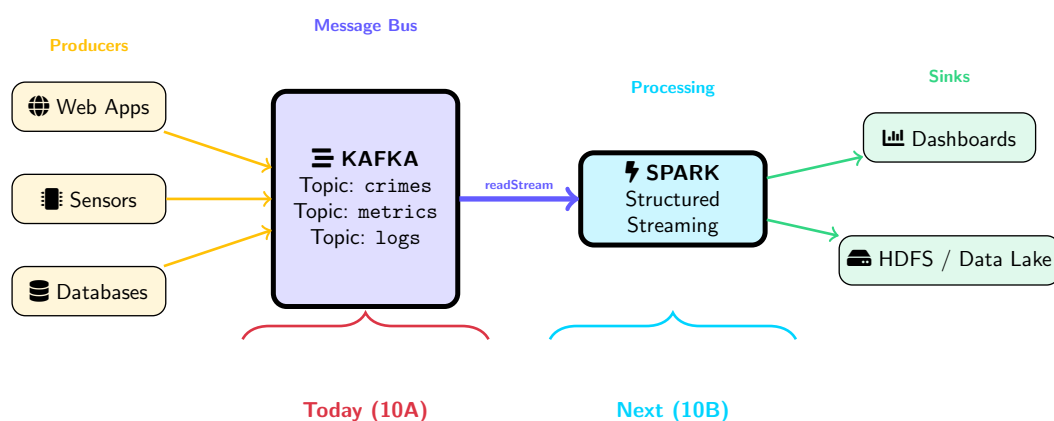
It acts as a **central hub** between data producers and data consumers.

Think of it as a **durable, replayable, high-speed message bus**.

Instructor Notes

Before we dive into architecture, let me give you the intuition. Imagine you are the CTO of Uber. Every second, 100,000 events happen across your platform: ride requests, GPS pings from drivers, payment confirmations, driver status changes. You need a system that can do four things simultaneously. First, receive all of those events without dropping a single one. Second, store them reliably on disk, not just in memory, because if a server crashes you cannot lose data. Third, let 10 different teams — the pricing team, the analytics team, the fraud detection team, the ETA team — all read the same events independently, at their own pace. Fourth, do all of this with millisecond latency. That system is Kafka. Now look at the definition at the bottom. Three keywords: distributed, meaning it runs on a cluster of machines. Fault-tolerant, meaning it survives server failures. Persistent, meaning messages are written to disk, not just held in memory. Think of it as a durable, replayable, high-speed message bus. Every word in that sentence matters, and we will unpack each one in the coming slides.

Where Kafka Fits in a Spark Architecture



The Key Idea: Decoupling

Producers and consumers are **completely independent**. A web app writes crime reports to Kafka. Spark reads them. Neither knows (or cares) about the other. You can add new producers or consumers at any time without changing anything.

Instructor Notes

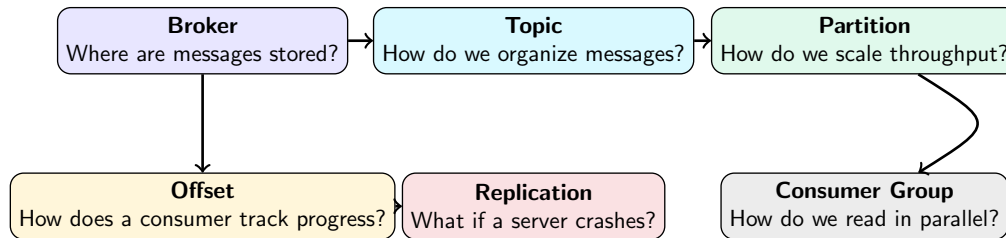
This is the architecture diagram I want you to memorize. Look at the flow from left to right. On the left, you have producers — web applications, sensors, databases — anything that generates data. They all write to Kafka, which sits in the middle as a message bus with topics like `crimes`, `metrics`, and `logs`. On the right of Kafka, Spark Structured Streaming reads those messages using `readStream`. After processing, the results go to sinks like dashboards or HDFS. Now look at the brackets at the bottom. The red bracket says “Today 10A” and covers only the Kafka portion. The blue bracket says “Next 10B” and covers the Spark processing portion. This is important: today we are only learning Kafka. Next session we connect the two. Now read the alert block. Producers and consumers are completely independent. The web app does not know that Spark exists. Spark does not know which web app produced the data. This is decoupling, and it is one of the most powerful properties of Kafka. You can add a new consumer tomorrow — say, a machine learning pipeline — without changing a single line of code in the producers.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Building Kafka's Architecture Step by Step

Instead of listing definitions, let us build the architecture from first principles.
For each concept, we ask: **what problem does it solve?**



The Hierarchy

Cluster $\supset$ Broker(s) $\supset$ Topic(s) $\supset$ Partition(s) $\supset$ Messages (with offsets)

Instructor Notes

I am going to teach you Kafka's architecture differently than most tutorials. Instead of dumping a table of definitions, we are going to build it from first principles. Each box on this diagram asks a question. Broker: where are messages physically stored? Topic: how do we organize them? Partition: how do we scale throughput? Offset: how does a consumer track where it left off? Replication: what if a server crashes? Consumer group: how do we read in parallel? The arrows show how these concepts depend on each other. You cannot understand partitions without first understanding topics. You cannot understand consumer groups without first understanding partitions. We will go through each one in order, and for each one, I will explain the problem it solves before defining it. Now look at the hierarchy at the bottom. A Kafka cluster contains brokers. Each broker contains topics. Each topic is divided into partitions. Each partition contains messages, and each message has an offset. This is the containment hierarchy you need to keep in your head as we go through the next several slides.

Brokers and Topics

Broker = A Kafka Server

- A single machine (or container) running Kafka
- Receives messages from producers
- Stores them on disk
- Serves them to consumers

Why multiple brokers?

One machine has finite disk and bandwidth. A *cluster* of brokers distributes the load.

Topic = A Named Category

- Like a database table or a folder
- Producers write to a topic by name
- Consumers subscribe to a topic by name

Example topics for our crime data:

`crimes` — all crime reports

`arrests` — arrest events only

`alerts` — real-time alerts

? Think About This

If a topic receives 1 million messages per second and lives on a single broker, what is the bottleneck? That single broker's disk I/O and network bandwidth. **This is why we need partitions.**

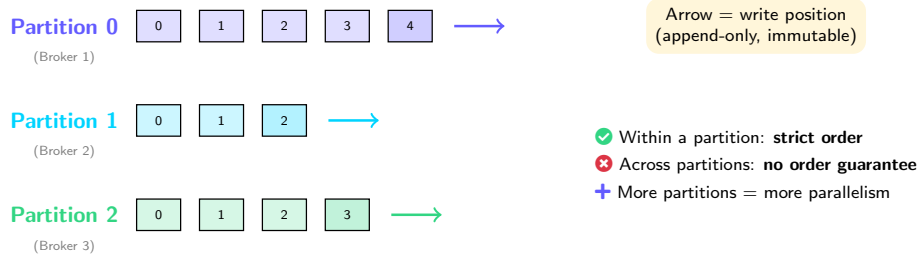
Instructor Notes

Let us start with the two simplest concepts. On the left: a broker. A broker is just a Kafka server — a single machine or container that receives messages, stores them on disk, and serves them to consumers. Think of it like a post office. Why do you need multiple brokers? Same reason you need multiple post offices in a city: one cannot handle all the mail. A cluster of brokers distributes the load. On the right: a topic. A topic is just a named category for messages. Think of it like a database table or a folder. You create a topic called `crimes`, and all crime reports go there. You create a topic called `arrests`, and arrest events go there. Producers write to a topic by name. Consumers subscribe to a topic by name. Very simple. Now I want you to think about the question at the bottom. If the `crimes` topic receives one million messages per second and it lives entirely on one broker, what is the bottleneck? That single broker's disk I/O and network bandwidth. One machine cannot write a million messages per second to disk. This is why we need the next concept: partitions.

Partitions — The Key to Kafka's Speed

A topic is split into **partitions** — *independent, ordered, append-only logs*.

Topic: “crimes” (3 partitions, spread across brokers)



Why Partitions Matter for Spark

Each Kafka partition maps to a **Spark task**.
3 partitions = 3 parallel Spark tasks.
This is how Spark achieves parallel reads.

Design Consequence

You can **increase** partition count later.
You **cannot decrease** it.
Choose wisely at creation time.

Instructor Notes

This is one of the most important slides in the entire lecture, so pay attention. A partition is an independent, ordered, append-only log. Look at the diagram. The topic “crimes” has been split into three partitions, and each partition lives on a different broker. Partition 0 is on Broker 1, Partition 1 on Broker 2, Partition 2 on Broker 3. Each partition is its own sequential log where messages get an offset number starting from zero. The arrow shows the write position — new messages are always appended at the end. Now look at the three rules on the right. Within a partition, messages are strictly ordered. If message 0 was written before message 1, a consumer will always read 0 before 1. But across partitions, there is no ordering guarantee. Message 0 in Partition 0 may have been written after message 2 in Partition 1. This is a trade-off: you gain parallelism but lose global ordering. Now look at the bottom left block. This is critical for next session: each Kafka partition maps to exactly one Spark task. Three partitions means three parallel Spark tasks reading from Kafka simultaneously. And the bottom right: you can increase partitions later, but you can never decrease them. So choose wisely when you create a topic.

How Messages Are Assigned to Partitions

No Key: Round-Robin

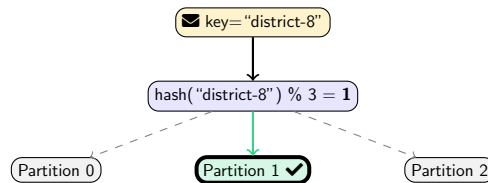
Messages spread evenly across all partitions.
Good for **load balancing** when order doesn't matter.

```
# No key -> round-robin
producer.produce("crimes",
    value="theft at district 8")
```

With Key: Hash Partitioning

$\text{hash}(\text{key}) \% \text{num_partitions}$
Same key → same partition **always**.

```
# With key -> hash to partition
producer.produce("crimes",
    key="district-8",
    value="theft at district 8")
```



⚠ Watch Out: Hot Partitions

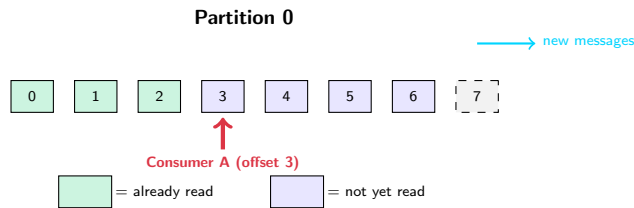
If you use district as the key and district 8 has 80% of all crimes, partition 1 gets 80% of the load. This is called a **hot partition** — the same skew problem we saw with Spark shuffles.

Instructor Notes

So a message needs to go to a partition. How does Kafka decide which one? Two strategies. On the left: if you do not provide a key, Kafka uses round-robin. Message one goes to Partition 0, message two to Partition 1, message three to Partition 2, then back to 0. This gives you perfect load balancing, but messages about the same entity can end up in different partitions, so you lose ordering for that entity. On the right: if you provide a key, Kafka hashes the key and takes the modulo with the number of partitions. Look at the diagram in the middle. The key is “district-8.” Kafka computes hash of “district-8” modulo 3, which equals 1. So this message goes to Partition 1. And here is the guarantee: every future message with key “district-8” will also go to Partition 1. Always. This gives you ordering per entity — all crimes from district 8 are in the same partition, in order. Now read the warning at the bottom. This is a trap that catches people. If district 8 has 80 percent of all crimes, then Partition 1 gets 80 percent of the load. This is a hot partition — the same skew problem you saw with Spark shuffles. In the lab, you will experiment with both strategies and see this firsthand.

Offsets — The Consumer's Bookmark

How does a consumer know where it left off? Every message gets a sequential **offset** within its partition.



Starting Positions

earliest: Read from offset 0 (replay all)
latest: Read only new messages

Retention

Kafka **retains** messages (default: 7 days).
Consumers can go back and re-read!

? What Happens If a Consumer Crashes?

Consumer A crashes after reading offset 2 but **before committing** offset 3. When it restarts, it re-reads from offset 2 — the last committed position. This is why Kafka's default delivery guarantee is **at-least-once**: safe, but you may see duplicates.

Instructor Notes

Now, how does a consumer know where it left off? Look at the diagram. We have Partition 0 with messages numbered 0 through 7. The green boxes are messages already read, the purple boxes are not yet read. The red arrow pointing at offset 3 says “Consumer A is here.” That offset is the consumer’s bookmark. Every time it reads a message, it advances its offset. Now look at the two starting positions at the bottom left. When a consumer first connects to a topic, it has two choices: **earliest** means start from offset 0 and read everything from the beginning. **latest** means skip everything that already exists and only read new messages going forward. For the lab, you will use **earliest** so you can replay data. In production, you typically use **latest**. Now read the crash scenario in the alert block. This is important. Consumer A reads offset 2, processes it, but crashes before committing offset 3. When it restarts, it starts from the last committed offset, which is 2. So it reads offset 2 again. This means you get a duplicate. This is why Kafka’s default guarantee is at-least-once: you never lose messages, but you might see some twice. We will talk about delivery guarantees in more detail later.

Kafka Retains Messages — It Is Not a Queue

This is the single most important difference between Kafka and traditional message queues.

Traditional Queue (RabbitMQ)

Consumer reads message → ACK → **deleted**
No replay. No second reader.
"Fire and forget"

Kafka Log

Consumer reads message → ACK → **still there**
Replay anytime. Multiple readers.
"Replayable event log"

Why Retention Matters — Three Concrete Scenarios

- 1 **New team joins:** The ML team wants to train on 30 days of crime data. They subscribe and read from **earliest** — no data pipeline needed.
- 2 **Bug fix replay:** Your Spark job had a bug. Fix it, reset the offset, **replay the same data** through the corrected pipeline.
- 3 **Multiple consumers:** The dashboard team AND the analytics team AND the alert system all read the **same messages** independently.

Instructor Notes

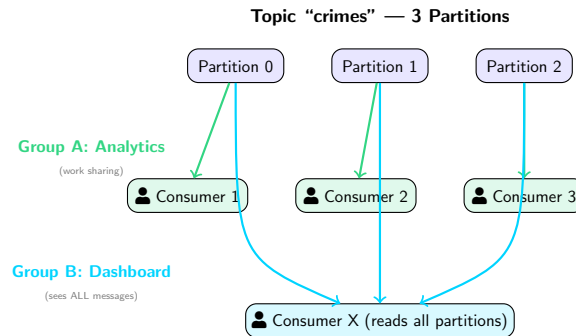
I need you to understand this slide deeply because it is the single most important difference between Kafka and every message queue you may have heard of. Look at the left box: a traditional queue like RabbitMQ. A consumer reads a message, acknowledges it, and the message is deleted. Gone. No other consumer can read it. No replay. Fire and forget. Now look at the right box: Kafka. A consumer reads a message, acknowledges it, and the message is still there. It stays on disk for a configurable retention period, default seven days. Any other consumer can read the same message. You can replay it. This changes everything. Let me give you three concrete scenarios. First, your ML team joins the project three weeks in. Instead of building a separate data pipeline, they just subscribe to the crimes topic with `auto.offset.reset=earliest` and read all 30 days of historical data. Second, your Spark streaming job had a bug. Fix the code, reset the consumer offset, and replay the same data through the corrected pipeline. No data loss. Third, the dashboard team, the analytics team, and the alert system all read the same messages independently. Each has its own consumer group, each tracks its own offsets. This is why Kafka is not a queue — it is a replayable event log.

Consumer Groups — Why Not Just One Consumer?

Scenario: the crimes topic receives 50,000 messages/sec. One consumer processes 5,000 msgs/sec. What happens?

Consumer lag grows by 45,000 messages every second. You fall further behind forever.

Solution: A *consumer group* — multiple consumers that **split the partitions** among themselves.



Rule

1 partition → at most

A. Koubaa (Alfaisal University)

Consumers > Partitions

Extra consumers sit **idle**

SE446 – Kafka

Two Groups

Each group gets **all**

Spring 2026

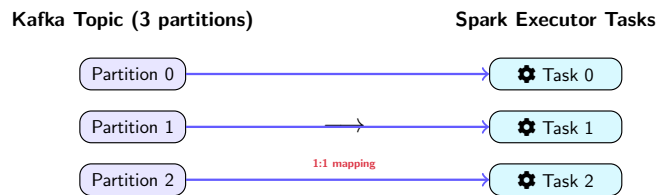
18 / 42

Instructor Notes

Let me start with the problem. Your `crimes` topic gets 50,000 messages per second. One consumer can process 5,000 per second. What happens? Every second, you fall 45,000 messages behind. That lag grows forever. You will never catch up. The solution is a consumer group: multiple consumers that share the work. Look at the diagram. Group A has three consumers, and the topic has three partitions. Kafka assigns one partition to each consumer. Consumer 1 reads Partition 0, Consumer 2 reads Partition 1, Consumer 3 reads Partition 2. Now you have three times the throughput. Now look at Group B at the bottom. It has only one consumer, Consumer X, which reads all three partitions by itself. Both groups see all messages — they are completely independent. Group A does not affect Group B and vice versa. This is the pub/sub pattern. Now look at the three rules at the bottom. First: one partition can only go to one consumer within a group. Second: if you have more consumers than partitions, the extra consumers sit idle with nothing to read. This is why partition count matters. Third: different groups are independent — each group gets all messages. In the lab, you will experiment with 1, 2, and 3 consumers and observe exactly this behavior.

Consumer Groups and Spark — The Connection

When Spark reads from Kafka, what happens under the hood?



What Spark Does Automatically

- Creates a **consumer group** named after your Spark application
- Assigns **one Spark task per Kafka partition** (just like one consumer per partition)
- Tracks offsets using **checkpointing** (Spark's equivalent of committing offsets)

💡 Practical Implication

The number of Kafka partitions **directly controls** Spark's read parallelism. If you have 3 partitions but 10 Spark executors, 7 executors sit idle during the read phase.

Instructor Notes

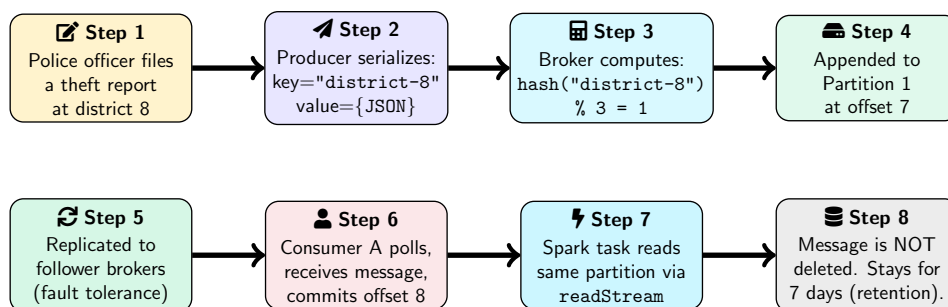
This slide connects everything you just learned about consumer groups to what you already know about Spark. Look at the diagram. On the left, three Kafka partitions. On the right, three Spark executor tasks. There is a 1-to-1 mapping. Partition 0 goes to Task 0, Partition 1 to Task 1, Partition 2 to Task 2. This is not a coincidence — this is how Spark reads from Kafka. Under the hood, when you write `spark.readStream.format("kafka")`, Spark creates a consumer group named after your application. It assigns one task per partition, exactly like assigning one consumer per partition in a consumer group. And instead of committing offsets like a regular Kafka consumer, Spark uses checkpointing — which is its own way of remembering where it left off. Now here is the practical implication that I want you to remember. The number of Kafka partitions directly controls Spark's read parallelism. If your topic has 3 partitions and you have 10 Spark executors, only 3 tasks will be active during the Kafka read phase. The other 7 executors have nothing to read. So when you create topics for your Milestone 5 pipeline, think about how many Spark tasks you want reading in parallel.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

A Crime Report's Journey Through Kafka

Let us trace a single crime event from the moment it happens to the moment Spark processes it.



Key Observations

The message is **immutable** once written. It is **replicated** for safety. Multiple consumers (a Python script, a Spark job, a dashboard) can all read it **independently** without interfering with each other.

Instructor Notes

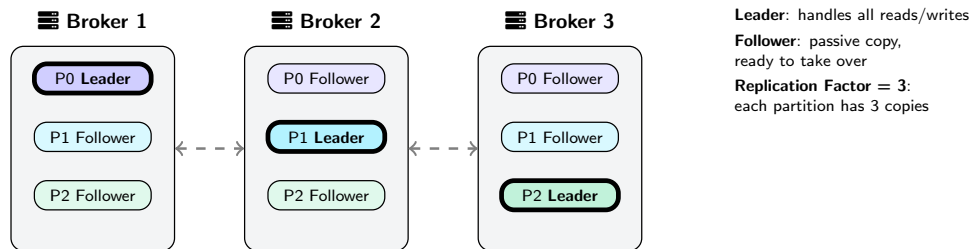
This is where everything comes together. Let us follow one crime event through the entire system. Step 1: a police officer files a theft report at district 8. Step 2: the producer application serializes this into a message with key “district-8” and value as a JSON payload. Step 3: the broker receives the message and computes hash of “district-8” modulo 3, which equals 1. So this message goes to Partition 1. Step 4: the message is appended to Partition 1 at offset 7 — the next available position. Now the second row. Step 5: the broker replicates this message to follower brokers on other machines for fault tolerance. Step 6: Consumer A polls Partition 1, receives the message at offset 7, processes it, and commits offset 8 as its new position. Step 7: independently, a Spark Structured Streaming task is also reading the same partition via `readStream` — it gets the same message. Step 8: the message is not deleted after being read. It stays on disk for seven days, the default retention period. Notice how Consumer A and the Spark task both read the same message without interfering with each other. That is the power of Kafka’s design — immutable, replicated, multi-consumer.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What’s Next

Replication — Kafka's Safety Net

What happens if the broker storing Partition 1 crashes? Without replication, all messages in that partition are **lost forever**.



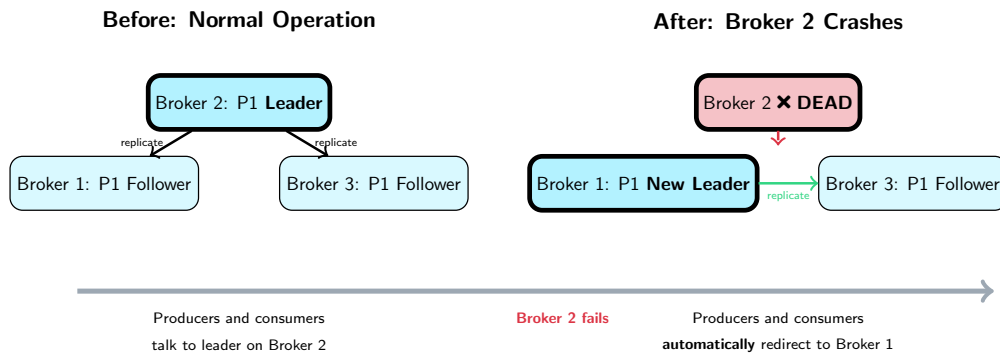
The Rule

Producers write to the **leader** only. Followers replicate asynchronously. If the leader dies, a follower is **automatically elected** as the new leader. As long as at least one replica is alive, **no data is lost**.

Instructor Notes

Now let us address the elephant in the room. What happens if a broker crashes? Without replication, all messages stored on that broker are lost forever. That is unacceptable for a production system. Look at this diagram carefully. We have three brokers. Each partition has three copies, but only one is the leader. The leader handles all reads and writes. The followers are passive copies that replicate data from the leader. Look at Broker 1: it is the leader for Partition 0, and it holds follower copies of Partitions 1 and 2. Broker 2 is the leader for Partition 1. Broker 3 is the leader for Partition 2. The replication factor is 3, meaning each partition exists on three different machines. Now read the rule at the bottom. Producers always write to the leader. Followers replicate asynchronously. If the leader dies, one of the followers is automatically elected as the new leader. As long as at least one replica is alive, no data is lost. In our course cluster, we only have one broker, so we use replication factor 1. But in production, you always use at least 3. This is what the `--replication-factor` flag means when you create a topic.

Broker Failure — What Actually Happens



Why This Matters

- Producers and consumers **do not need to change their code**. Kafka handles failover.
- With `replication-factor=3`, Kafka tolerates **2 simultaneous broker failures**.
- This is why `--replication-factor` is a required parameter when creating topics.

Instructor Notes

Let us walk through exactly what happens when a broker dies. Look at the left side: normal operation. Broker 2 is the leader for Partition 1. It replicates data to followers on Broker 1 and Broker 3. Everything is fine. Now look at the right side: Broker 2 crashes. What happens? Kafka detects the failure, and one of the followers — in this case Broker 1 — is automatically elected as the new leader for Partition 1. Broker 3 remains a follower and now replicates from Broker 1 instead. Look at the timeline at the bottom. Before the failure, producers and consumers talk to the leader on Broker 2. After the failure, they automatically redirect to Broker 1. They do not need to change any code, any configuration, anything. Kafka handles the failover transparently. Now here is the key math: with a replication factor of 3, each partition has 3 copies. That means Kafka can tolerate 2 simultaneous broker failures and still lose no data. For your lab, we use replication factor 1 because we only have one broker. But in production, always use at least 3. This is non-negotiable for any serious deployment.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Kafka vs RabbitMQ — Different Tools for Different Problems

Feature	Apache Kafka	RabbitMQ
Mental model	Append-only log	Work queue
Message after read	✓ Retained on disk	✗ Deleted after ACK
Replay	Reset offset → re-read	Not possible
Throughput	1M+ msgs/sec per broker	~50K msgs/sec
Ordering	Guaranteed per partition	Guaranteed per queue
Multiple readers	Independent consumer groups	Message consumed once
Best for	Event streaming, log aggregation	Task distribution, RPC

⚠ Important Distinction

Kafka is **NOT** a replacement for RabbitMQ. Different tools for different problems.

Kafka = “What happened in my system?” (event log, many readers)

RabbitMQ = “Do this job for me” (task queue, one worker picks it up)

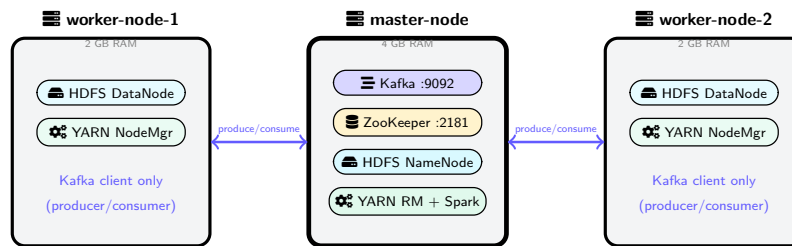
Instructor Notes

I get this question every semester: should I use Kafka or RabbitMQ? The answer is: they solve different problems. Let me walk through the key rows. Mental model: Kafka is a log — an append-only, persistent record of events. RabbitMQ is a queue — a pipe that delivers tasks to workers. Message after read: in Kafka, the message stays on disk. In RabbitMQ, it is deleted after acknowledgement. Replay: in Kafka, you reset the offset and re-read. In RabbitMQ, once consumed, the message is gone. Throughput: Kafka handles over a million messages per second per broker. RabbitMQ handles about 50,000. That is a 20x difference. Ordering: Kafka guarantees order per partition. RabbitMQ guarantees order per queue. Multiple readers: Kafka supports independent consumer groups, each reading all messages. RabbitMQ delivers each message to exactly one consumer. Now read the alert block. This is important: Kafka is not a replacement for RabbitMQ. If you need to answer “what happened in my system” with multiple teams reading the same events, use Kafka. If you need to say “do this job for me” and have one worker pick it up, use RabbitMQ. Different tools for different problems.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice**
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Our Cluster — Three Nodes, One Kafka Broker



Connection Info

`bootstrap.servers = master-node:9092`
Use this in all producers and consumers.

Security

Port 9092 is **firewalled** — only cluster nodes and campus network can connect. ZooKeeper (2181) is localhost only.

Instructor Notes

Let me show you the actual cluster you will be using for the lab and for Milestone 5. Look at the three boxes. In the center is the master node with 4 gigabytes of RAM. This is where Kafka runs — the broker on port 9092 and ZooKeeper on port 2181. It also hosts the HDFS NameNode, the YARN ResourceManager, and Spark. On the left and right are the two worker nodes, each with 2 gigabytes of RAM. They run HDFS DataNodes and YARN NodeManagers for Spark executors. For Kafka, the workers are clients only — they can produce and consume messages, but the broker itself runs only on the master. The arrows show that workers communicate with the master's Kafka broker to send and receive messages. Now look at the bottom. The connection string is `bootstrap.servers = master-node:9092`. Write this down — you will use it in every producer and consumer you write. On the right, a security note: port 9092 is firewalled to only allow connections from cluster nodes and the campus network. ZooKeeper on port 2181 is restricted to localhost only. This means random internet users cannot connect to our Kafka broker.

Our Cluster — Software Stack

Component	Version	Role in Streaming Pipeline
Hadoop / HDFS	3.4.1	Distributed storage (data lake sink)
YARN	3.4.1	Resource management for Spark executors
Spark	3.5.4	Batch + streaming processing engine
Hive	4.0.1	SQL queries on stored data
Kafka	3.9.0	Real-time message ingestion (NEW)
ZooKeeper	(bundled)	Broker coordination (NEW)
Java	OpenJDK 11	Runtime for all services
Python	3.10+	PySpark + confluent-kafka client

The Full Pipeline You Are Building (M5)

Python Producer $\xrightarrow{\text{port 9092}}$ **Kafka Topic** $\xrightarrow{\text{readStream}}$ **Spark Streaming** $\rightarrow$ **HDFS / Console**

Everything runs on this cluster. No external services needed.

Instructor Notes

This table shows the complete software stack running on our cluster. I want you to see how everything fits together. Hadoop and HDFS at version 3.4.1 provide the distributed storage layer — this is where your processed data can be written as the final sink. YARN manages resources and allocates executors for Spark jobs. Spark 3.5.4 is our processing engine, and starting today, it will also handle streaming workloads. Hive is there for SQL queries on stored data. Now look at the two highlighted rows: Kafka 3.9.0 and ZooKeeper are the new additions we just installed. Kafka handles real-time message ingestion, and ZooKeeper coordinates the broker. All of this runs on Java 11, and you interact with it through Python using PySpark and the `confluent-kafka` library. Now look at the pipeline at the bottom. This is exactly what you are building for Milestone 5. A Python producer writes crime events to a Kafka topic on port 9092. Spark Structured Streaming reads from that topic using `readStream`. The processed results go to HDFS or the console. Everything runs on this cluster — no external services, no cloud accounts, nothing else needed. Today you build the first half; next session you complete it.

CLI Demo — Create, Produce, Consume

```
# Step 1: Create a topic with 3 partitions
kafka-topics.sh --create --topic demo-crimes \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1

# Step 2: List all topics
kafka-topics.sh --list --bootstrap-server localhost:9092

# Step 3: Start a consumer (Terminal 1 -- listening)
kafka-console-consumer.sh --topic demo-crimes \
  --bootstrap-server localhost:9092

# Step 4: Produce messages (Terminal 2 -- typing)
kafka-console-producer.sh --topic demo-crimes \
  --bootstrap-server localhost:9092
> {"type":"THEFT","district":8,"hour":14}
> {"type":"BATTERY","district":11,"hour":22}
```

What You'll See

As soon as you type a message in Terminal 2, it appears **instantly** in the consumer in Terminal 1. This is real-time messaging in action.

Instructor Notes

Let me walk through this CLI demo step by step because you will do exactly this in the lab. Step 1: create a topic called `demo-crimes` with 3 partitions and replication factor 1. Notice the flag `--partitions 3` — this determines how many parallel consumers can read from this topic. Replication factor 1 because we only have one broker. Step 2: list all topics to confirm it was created. Step 3 is where it gets interesting. Open one terminal and start a console consumer. This consumer will sit there and wait for messages. It is listening. Step 4: open a second terminal and start a console producer. Now type a JSON message and hit enter. The moment you press enter, that message appears instantly in the consumer terminal. This is real-time messaging. There is no batch job, no waiting, no file scan. The message travels from your producer through the Kafka broker to the consumer in milliseconds. In the lab, you will do this with two SSH sessions to the cluster, and I want you to experience that instant delivery firsthand.

Python Producer — Annotated

Each line has a purpose. Let us walk through them.

```
from confluent_kafka import Producer      # (1) Kafka Python client
import json, time

conf = {'bootstrap.servers': 'MASTER_IP:9092'} # (2) Broker address
producer = Producer(conf)                 # (3) Create producer instance

crimes = [
    {"id": 1, "type": "THEFT", "district": 8, "arrest": False},
    {"id": 2, "type": "BATTERY", "district": 11, "arrest": True},
    {"id": 3, "type": "ROBBERY", "district": 3, "arrest": False},
]

for crime in crimes:
    producer.produce(
        topic="crimes",                    # (4) Which topic
        key=str(crime["district"]),        # (5) Partition key
        value=json.dumps(crime).encode("utf-8") # (6) Payload as bytes
    )
    print(f"Produced: {crime['type']} in district {crime['district']}")
    time.sleep(1)

producer.flush() # (7) Block until ALL messages are delivered
```

Why These Choices Matter

- (5) Key = district → all crimes from the same district go to the same partition → **ordering per district**.
- (6) Kafka stores bytes, not objects. You must serialize (and deserialize on the consumer side).
- (7) produce() is **async** (returns immediately). Without flush(), your program may exit before messages are sent.

Navigation icons: back, forward, search, etc.

Instructor Notes

Let me walk through this Python producer line by line. Line 1, we import the `Producer` class from the `confluent_kafka` library. This is the official Kafka Python client. Line 4, we create a configuration dictionary with the broker address. Line 5, we create a producer instance. Nothing has been sent yet. Now look at the loop. For each crime, we call `producer.produce()` with three arguments. Argument 4 is the topic name: “crimes.” Argument 5 is the key: the district number as a string. Remember what keys do — all crimes from district 8 will always go to the same partition, giving you ordering per district. Argument 6 is the value: the crime data serialized as JSON and encoded to bytes. This is critical — Kafka stores raw bytes, not Python objects. You must serialize on the producer side and deserialize on the consumer side. Now the most important line is line 7: `producer.flush()`. The `produce()` call is asynchronous — it returns immediately and queues the message in a buffer. Without `flush()`, your program could exit before the messages are actually sent. Every semester I get students who forget flush and wonder why their consumer sees nothing. Do not be that student.

Python Consumer — Annotated

```
from confluent_kafka import Consumer
import json

conf = {
    'bootstrap.servers': 'MASTER_IP:9092',
    'group.id': 'crime-analytics-group', # (1) Consumer group name
    'auto.offset.reset': 'earliest'      # (2) Start position
}

consumer = Consumer(conf)
consumer.subscribe(['crimes'])          # (3) Which topic(s) to read

try:
    while True:
        msg = consumer.poll(timeout=1.0) # (4) Pull model: consumer asks
        if msg is None:                  # for messages (not pushed)
            continue
        if msg.error():
            print(f"Error: {msg.error()}")
            continue
        crime = json.loads(msg.value().decode("utf-8")) # (5) Deserialize
        print(f"Type: {crime['type']} | Partition {msg.partition()} | "
              f"Offset {msg.offset()}")
finally:
    consumer.close()                    # (6) Cleanly leave the group
```

Key Design Decisions

- (1) `group.id`: consumers with the same group ID share partitions. Different group ID = independent copy of all messages.
- (2) `earliest` = replay from the beginning. `latest` = only new messages.
- (4) `poll()`: Kafka uses a **pull model** — the consumer controls its own pace. This prevents slow consumers from being overwhelmed.

Instructor Notes

Now the consumer side. Look at the configuration. Three settings. First, `group.id`: this is the consumer group name. All consumers with the same group ID share partitions. If you want a second team to independently read all messages, give them a different group ID. Second, `auto.offset.reset`: set to `earliest`, which means if this consumer group has never read this topic before, start from the very beginning. If you set it to `latest`, it would skip all existing messages and only read new ones. For the lab, always use `earliest` so you can replay. Third, we subscribe to the “crimes” topic. Now look at the main loop. The key is `consumer.poll(timeout=1.0)` at line 4. This is the pull model. The consumer asks Kafka for messages, not the other way around. If there are no messages, `poll()` returns `None` after one second, and we continue. If there is a message, we deserialize it from bytes to JSON at line 5, then print the crime type, partition number, and offset. At line 6, `consumer.close()` cleanly leaves the consumer group so Kafka can reassign partitions. If you kill the process without closing, Kafka has to wait for a timeout before reassigning, which delays other consumers.

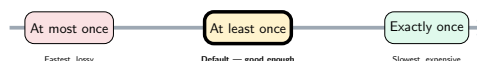
Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Delivery Guarantees — What Can Go Wrong?

When sending a message from producer to consumer, three things can go wrong: *loss, duplication, or both.*

Guarantee	Meaning	Kafka Config	Scenario
At most once (loss is OK)	May lose, never duplicate	acks=0	Log metrics
At least once (duplicates handled)	Never lose, may duplicate	acks=all	Crime reports
Exactly once (must be exact)	Neither lost nor duplicated	Idempotent + transactions	Bank transfers



Practical Advice

Most production systems use **at-least-once** with **idempotent consumers** (detecting and ignoring duplicates). Exactly-once adds significant overhead. **Spark Structured Streaming + Kafka checkpointing achieves exactly-once end-to-end** — covered in Session 10B.

Instructor Notes

There are three levels of delivery guarantees, and each has a trade-off. At most once: the producer sends a message and does not wait for acknowledgement. This is the fastest because there is no round trip, but if the broker fails, the message is lost. Use case: log metrics where losing a few data points is acceptable. At least once: the producer waits for acknowledgement. If it does not receive one, it retransmits. This guarantees no data loss, but you might get duplicates if the acknowledgement was lost while the message was actually stored. This is the default and what most systems use. Use case: crime reports, where duplicates can be detected and filtered. Exactly once: neither lost nor duplicated. This requires idempotent producers and transactions, which add significant overhead. Use case: bank transfers where processing a payment twice is unacceptable. Look at the spectrum at the bottom. At most once is fastest but lossy. Exactly once is safest but expensive. At least once is the sweet spot for most real systems. And here is the bridge to next session: Spark Structured Streaming combined with Kafka checkpointing can achieve exactly-once end-to-end semantics. We will see exactly how in Session 10B.

How Many Partitions? — A Design Checklist

This is one of the first decisions you make when creating a topic. How do you choose?

Factor	Recommendation
Consumer parallelism	At least as many partitions as max consumers in a group. Also: at least as many as Spark executor cores.
Expected throughput	More partitions = higher throughput (each = sequential I/O)
Ordering requirements	Fewer partitions if strict global order matters. 1 partition = total order (but no parallelism).
Key cardinality	If using keys, ensure enough distinct keys to spread evenly
Cluster limits	Don't exceed ~10K partitions per broker

💡 Rule of Thumb

Start with the **number of consumers (or Spark cores) you expect**, then **double it** for future scaling. You can increase partitions later, but you **cannot decrease** them.

Instructor Notes

When you create a topic, the first question is: how many partitions? Let me walk through the decision factors. Consumer parallelism: you need at least as many partitions as consumers in a group. For Spark, that means at least as many partitions as Spark executor cores reading from Kafka. If you have 6 cores and 3 partitions, half your cores sit idle. Expected throughput: each partition is a separate sequential I/O stream on the broker. More partitions means more parallel I/O, means higher throughput. Ordering: this is the trade-off. If you need strict global ordering across all messages, you need exactly 1 partition. But then you lose all parallelism. In practice, you use keys to get per-entity ordering within partitions, which is usually good enough. Key cardinality: if you use keys, make sure you have enough distinct keys. If all your messages have the same key, they all go to one partition, defeating the purpose. And the rule of thumb at the bottom: start with the number of consumers or Spark cores you expect, then double it for future growth. Remember, you can increase partitions later but you can never decrease them. For your Milestone 5 topic, I recommend starting with 3 partitions — matches our cluster size.

Retention Policy — How Long Does Kafka Keep Messages?

```
# Time-based: keep messages for 7 days (default)
log.retention.hours=168

# Size-based: keep up to 1 GB per partition
log.retention.bytes=1073741824

# Compaction: keep only latest value per key
log.cleanup.policy=compact
```

Time-Based (Most Common)

Messages older than `retention.hours` are deleted.
Configurable per topic; 7 days is the default.

Compaction (Advanced)

Only keeps the **latest value** per key.
Useful for “current state” lookups
(e.g., latest status of each district).

Why This Matters for Your Spark Pipeline

If your Spark job goes down for 3 days and Kafka retains data for 7 days, you can restart and **replay everything you missed**.
If retention is only 1 day, those 2 extra days of data are **gone**.

Instructor Notes

How long does Kafka keep messages? There are three options in the configuration. Time-based: the default is 168 hours, which is 7 days. After that, messages are deleted automatically. Size-based: you can also set a maximum size per partition. When the partition exceeds that size, old messages are removed. And compaction: this is an advanced mode where Kafka keeps only the latest value per key. For example, if district 8 sends status updates every minute, compaction keeps only the most recent one. For this course, we use the default time-based retention. Now here is the practical scenario I want you to think about. Your Spark Streaming job crashes on a Friday night and nobody notices until Monday. That is roughly 3 days of downtime. If Kafka retention is 7 days, you restart Spark on Monday and it replays all the messages it missed — no data loss. But if someone had set retention to 1 day, those 2 extra days of data are permanently gone. This is why retention settings matter for your pipeline's resilience. For Milestone 5, the default 7-day retention is more than sufficient.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World**
- 9 Summary and What's Next

Who Uses Kafka and Why?

Company	Use Case	Scale
LinkedIn	Activity tracking, metrics	7 trillion msgs/day
Netflix	Real-time recommendations	1 PB/day
Uber	Trip events, pricing, matching	Millions of msgs/sec
Twitter/X	Stream processing, analytics	Global event bus

Why Kafka Won

No other system combined these four properties at the time:

High Throughput
1M+ msgs/sec

Fault Tolerance
replication

Persistence
disk-backed log

Decoupling
pub/sub model

Fun Fact

Kafka was created at **LinkedIn** in 2011, open-sourced via Apache. Named after the author Franz Kafka — because it was designed for **writing** (data).

Instructor Notes

Let me give you a sense of scale. LinkedIn, where Kafka was created, processes 7 trillion messages per day through Kafka. That is 7 with 12 zeros. Netflix pushes 1 petabyte of data per day through Kafka for real-time recommendation updates. Uber routes millions of messages per second — every GPS ping from every driver, every ride request, every payment — all through Kafka. Now look at the four properties at the bottom that explain why Kafka won. High throughput: over a million messages per second per broker. Fault tolerance: replication across brokers ensures no data loss. Persistence: unlike in-memory systems, Kafka writes to disk, so data survives restarts. Decoupling: the pub/sub model means producers and consumers are independent. Before Kafka, no system combined all four. You could have high throughput without persistence, or persistence without throughput. Kafka was the first to do all four. And a fun fact: Kafka was named after the author Franz Kafka, because the system was designed for writing — specifically, writing massive amounts of data. It was created at LinkedIn in 2011 and open-sourced through the Apache Software Foundation, which is the same foundation that gave us Hadoop, Spark, and Hive.

Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

Key Takeaways

- 1 **Why Kafka?** The $N \times N$ integration problem + batch latency. Kafka provides a **central, persistent, high-throughput message bus**.
- 2 **Architecture:** Cluster → Broker → Topic → Partition → Offset. Messages are **ordered within a partition**, retained on disk, replicated for safety.
- 3 **Partitions** are the unit of parallelism — both for Kafka consumers **and** for Spark tasks that read from Kafka.
- 4 **Consumer Groups** enable parallel reading (work sharing within a group) and independent reading (different groups = pub/sub).
- 5 **Not a queue:** Messages are **retained, not deleted**. This enables replay, multiple consumers, and crash recovery.
- 6 **Fault tolerance:** Replication across brokers ensures no data loss even when machines fail.

Core Formula

Producers → Kafka (Topics/Partitions) → Consumer Groups

Instructor Notes

Let me summarize the six key things I want you to take away from today. First, why Kafka exists: it solves the N-times-N integration nightmare and the batch latency problem. Second, the architecture hierarchy: cluster, broker, topic, partition, offset. Messages are ordered within a partition, retained on disk, and replicated across brokers. Third, partitions are the unit of parallelism. This matters for Kafka consumers and for Spark tasks — each Kafka partition maps to one Spark task. Fourth, consumer groups. Within a group, consumers share partitions for work splitting. Across groups, each group independently reads all messages for the pub/sub pattern. Fifth, Kafka is not a queue. Messages are retained, not deleted after reading. This enables replay, multiple consumers, and crash recovery. Sixth, fault tolerance through replication. As long as one replica is alive, no data is lost. And the core formula at the bottom: producers write to Kafka topics and partitions, consumer groups read from them. This is the entire system in one line. Everything we covered today is a detail of this formula.

Sneak Peek — Connecting Kafka to Spark (Session 10B)

Today we built the highway. Next session, we build the cars that drive on it.

This is all it takes to connect Spark to the Kafka topics you created today:

```
# Read a stream of crime reports from Kafka
stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "MASTER_IP:9092") \
    .option("subscribe", "crimes") \
    .load()
```

What Happens Under the Hood

- Spark creates a **consumer group** and assigns one task per partition
- Each Kafka message arrives as a row with columns: key, value, topic, partition, offset, timestamp
- You parse value (bytes) into a structured DataFrame — same API you already know
- Spark tracks offsets via **checkpointing** — if it crashes, it resumes exactly where it left off

Instructor Notes

I want to end with a preview of what comes next, because I want you to see that everything you learned today has a direct payoff. Look at this code. Five lines. That is all it takes to connect Spark to Kafka. `spark.readStream` tells Spark you want a streaming read, not a batch read. `format("kafka")` says the source is Kafka. The options specify the broker address and the topic name. And `load()` starts the connection. Under the hood, Spark creates a consumer group, assigns one task per Kafka partition — exactly what we discussed today — and each message arrives as a row with columns for key, value, topic, partition, offset, and timestamp. The value is still raw bytes, so you need to deserialize it just like in the Python consumer. And Spark tracks offsets via checkpointing, so if the job crashes, it resumes exactly where it left off. This is the exactly-once guarantee I mentioned earlier. Next session, we will build a complete pipeline: read crime data from Kafka, parse the JSON, compute aggregations like crimes per district per hour, and write the results to a dashboard. Everything you learned today is the foundation for that.

What's Next

Session 10B: Spark Structured Streaming

- The “infinite table” model — process streams with the DataFrame API you already know
- `readStream` from Kafka → parse JSON → aggregate → `writeStream`
- Output modes: append, update, complete
- Time windows: tumbling and sliding
- Watermarks for handling late data

Today's Lab: Kafka Fundamentals

- Create topics, produce crime messages with Python
- Observe partition assignment with and without keys
- Experiment with consumer groups (1, 2, 3 consumers)
- Monitor consumer lag

Milestone M5 Reminder

Due Week 12: End-to-end streaming pipeline (Kafka → Spark Streaming). Today you build the Kafka foundation; next session you complete the pipeline.

Instructor Notes

Here is what is coming. Session 10B covers Spark Structured Streaming, where you will learn the “infinite table” model — treating a stream as a table that keeps growing. You will use the same DataFrame API you already know: `select`, `filter`, `groupBy`, `agg`. The only difference is that you read with `readStream` instead of `read` and write with `writeStream` instead of `write`. We will cover output modes, time windows, and watermarks for handling late data. But first, today’s lab. You are going to create Kafka topics, produce crime messages using the Python producer we just saw, observe how partition assignment works with and without keys, experiment with consumer groups by running 1, 2, then 3 consumers simultaneously, and monitor consumer lag. This lab builds the foundation you need for Milestone 5, which is due in Week 12: a complete end-to-end streaming pipeline from Kafka to Spark. Today you build the Kafka side. Next session you build the Spark side. Then you connect them for M5. Any questions before we move to the lab?